



By Nitesh Singh

Job Ready Free Live Training To Become A

SOC

Analyst L1 For Free

Defronix Cyber Security → <https://defronix.com>

LINUX FILE SYSTEM STRUCTURE



WHAT IS LINUX FILE SYSTEM?

Linux organizes everything in a **single hierarchical tree structure**.

Unlike Windows:

C:\ D:\ E:\

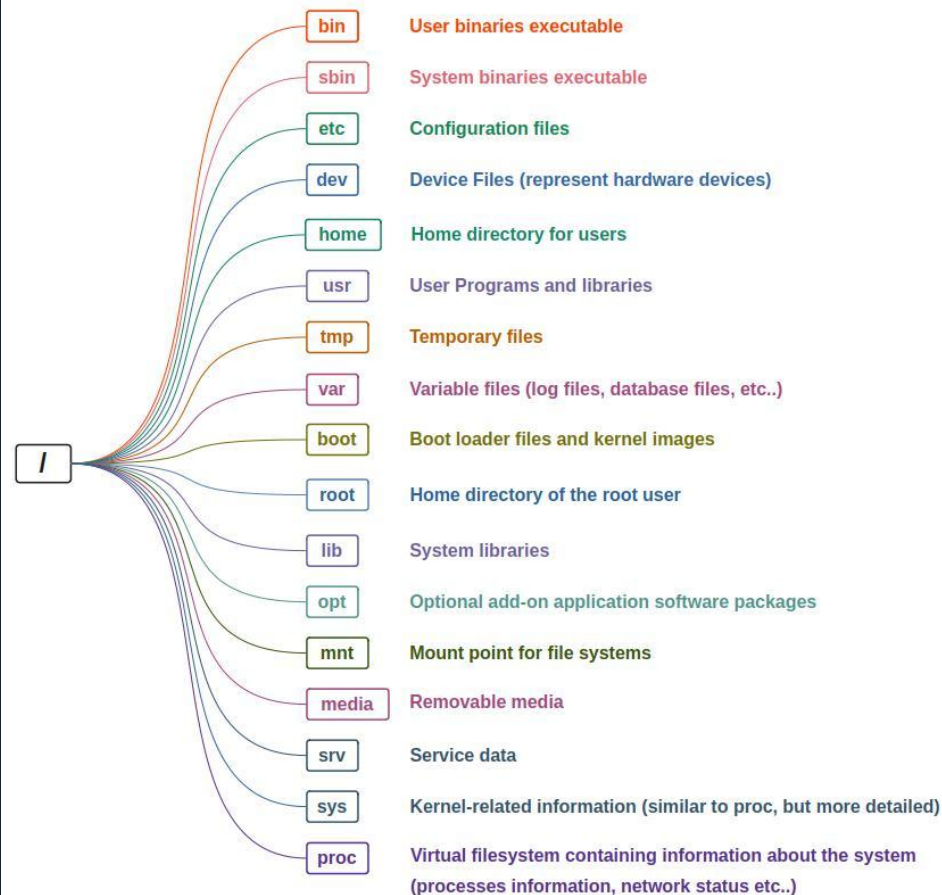
Linux starts from:

/

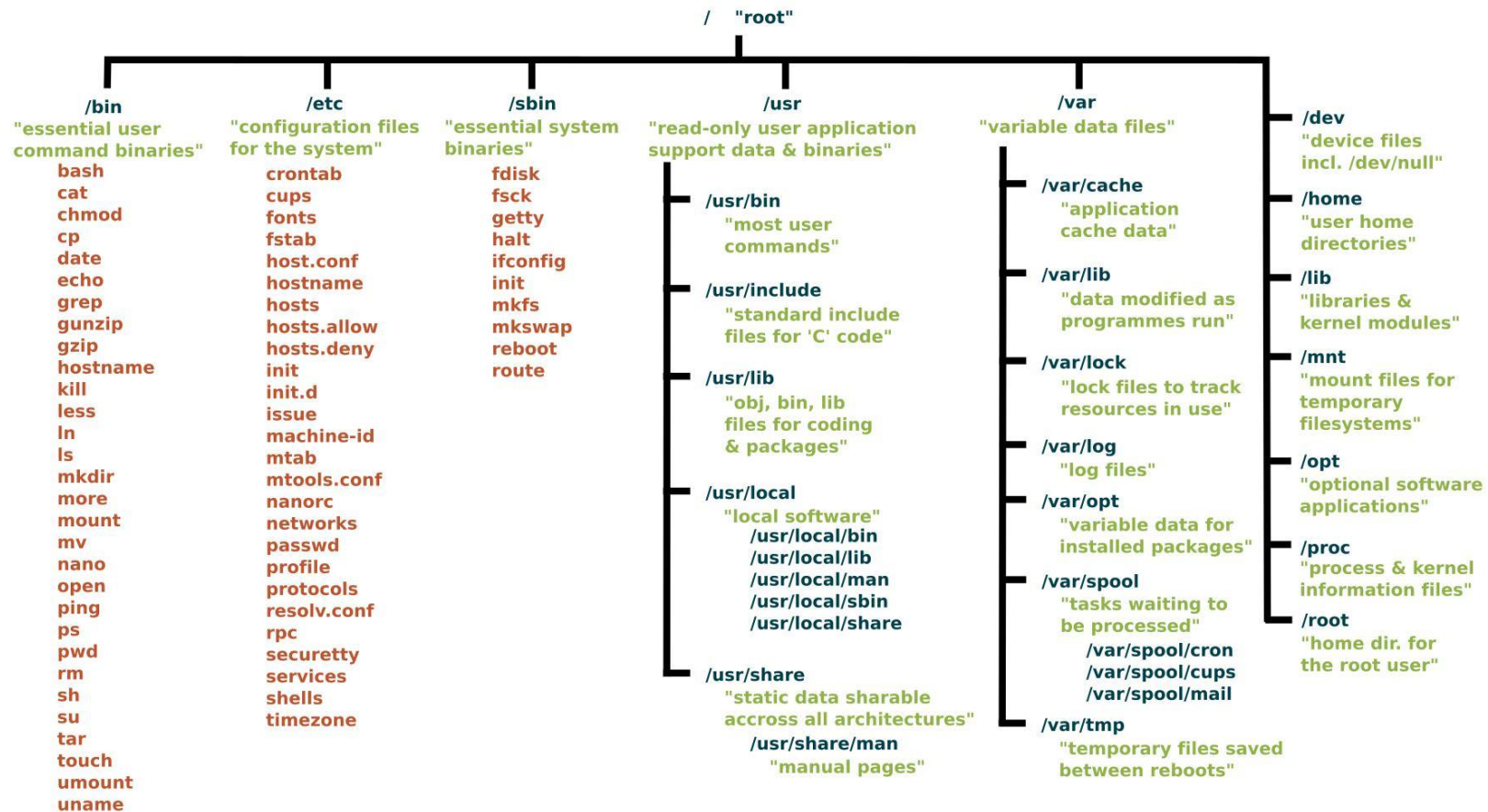
This is called: **ROOT DIRECTORY**

Everything starts from root.

LINUX FILE SYSTEM STRUCTURE



LINUX FILE SYSTEM STRUCTURE



LINUX PERMISSIONS



In Linux, everything is a file, and every file has an owner and a group. Permissions define how the **Owner**, the **Group**, and **Everyone Else** can interact with those files.

Read, Write, and Execute

Permission	Symbol	File Meaning	Directory Meaning
Read	r	View the file contents.	List the files inside (ls).
Write	w	Modify the file contents.	Create, delete, or rename files inside.
Execute	x	Run the file as a program.	"Enter" the directory (cd).

Note: To delete a file, you actually need **Write** and **Execute** permissions on the *parent directory*, not necessarily the file itself.

LINUX PERMISSIONS



When you run `ls -l`, you see a string like `-rwxr-xr--`. Let's dissect that:

1. **The First Character:** Indicates the file type (`-` for regular file, `d` for directory, `l` for symbolic link).
2. **Sets of Three:** The next nine characters are divided into three triplets:
 - **User (u):** The individual who owns the file.
 - **Group (g):** Members of the group assigned to the file.
 - **Others (o):** Everyone else on the system.

Example: `-rwxr-xr--`

- **User:** `rwx` (Read, Write, Execute)
- **Group:** `r-x` (Read, Execute)
- **Others:** `r--` (Read only)



Numeric (Octal) Notation

System admins rarely type out rwx. Instead, we use a base-8 system. Each permission is assigned a value:

- **Read (r)** = 4
- **Write (w)** = 2
- **Execute (x)** = 1
- **None (-)** = 0

To get the number for a triplet, you just add them up.

$$4 \text{ (r)} + 2 \text{ (w)} + 1 \text{ (x)} = 7$$

LINUX PERMISSIONS



Numeric (Octal) Notation

Total	Permissions	Description
7	<code>rwX</code>	Full control
6	<code>rw-</code>	Read and write
5	<code>r-X</code>	Read and execute
4	<code>r--</code>	Read-only
0	<code>---</code>	No access



What is /var/log?

Main Linux logging directory.

Contains:

- Authentication logs
- System logs
- Application logs

Why SOC cares

Linux investigations = log analysis.



/var/log/auth.log 🔥

MOST IMPORTANT.

Contains:

- SSH logins
- Authentication attempts
- sudo usage

/var/log/syslog

General system logs.

/var/log/kern.log

Kernel logs.

/var/log/secure

Used in RedHat/CentOS.

Equivalent of:



/var/log/messages

General system activity.

/var/log/apache2/access.log

Web access logs.

/var/log/apache2/error.log

Web server errors.



What are Authentication Logs?

Track:

- User logins
- SSH access
- Failed logins
- sudo usage

Main Files

OS	File
Ubuntu/Debian	<code>/var/log/auth.log</code>
RHEL/CentOS	<code>/var/log/secure</code>

Example Log

```
Jul 10 10:22:11 server sshd[2345]:  
Accepted password for admin from 192.168.1.10
```



What is SSH?

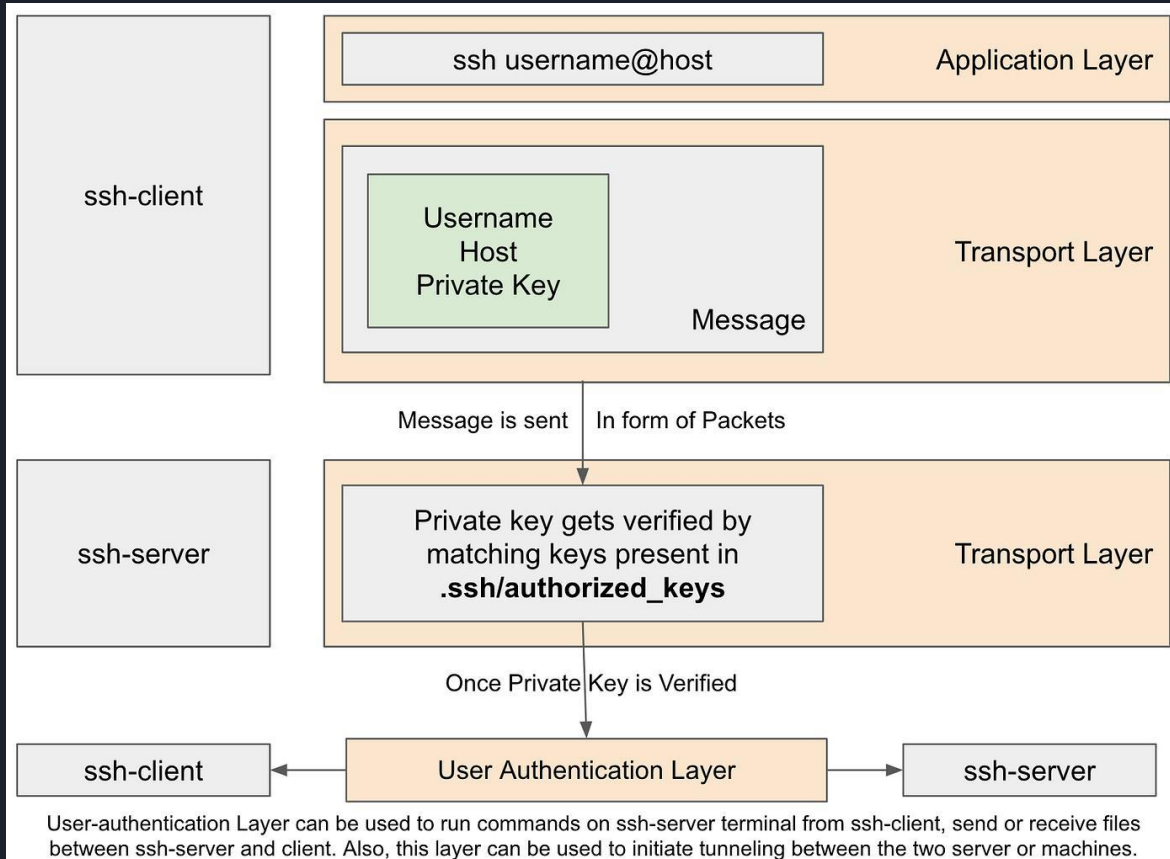
Secure Shell:

- Remote Linux access
- Port 22

Why attackers love SSH

- Remote control
- Persistence
- Lateral movement

LINUX SSH LOGS





ssh

Client command.

sshd

SSH daemon (server process).

Port 22

Default SSH port.

SSH AUTH METHODS

Password Authentication

Username + password.

Key-Based Authentication

More secure.

Uses:

- Public key
- Private key



Password Authentication (The Basic Way)

The client sends the username and password through the encrypted tunnel. The server checks **/etc/shadow** (on Linux) to see if they match. Simple, but vulnerable to brute-force attacks.

2. Public Key Authentication (The Secure Way)

This uses **Asymmetric Encryption** (RSA or Ed25519). This is the most common professional method.

- **The Setup:** You have a **Private Key** (kept on your machine) and a **Public Key** (placed in the server's **~/.ssh/authorized_keys** file).
- **The Challenge:**
 1. The client tells the server which key it wants to use.
 2. The server generates a random number (a "challenge"), encrypts it using your **Public Key**, and sends it to the client.
 3. Only your **Private Key** can decrypt that message. The client decrypts it, combines it with the session ID, and sends a "signature" back.
 4. The server verifies the signature. If it matches, you're in.

SSH Config : /etc/ssh/sshd_config



SOC Importance

Attackers add their own SSH keys for persistence.

REAL SOC INVESTIGATION FLOW

Scenario

Many failed SSH logins
Then success
Then sudo activity

SOC Thinking

Brute force → compromise → privilege escalation



FINAL SOC MINDSET

When investigating Linux:

Don't ask:

✗ "Was there a login?"

Ask:

- Who logged in?
- From where?
- Using what method?
- What happened after login?
- Was privilege escalation used?